

College Football Conference Analytics Dashboard

Project Proposal

Developer	Matt Kurtz
Course	MSBA-692
Target Audience	College football fans, sports analysts, researchers, and students interested in sports analytics
Technology Stack	Python, Dash, Plotly, Pandas, SQLAlchemy, CollegeFootballData API, Supabase PostgreSQL

Background

This project develops an interactive college football analytics dashboard focused on SEC and Big Ten programs. Data is extracted from the CollegeFootballData API, transformed through a Python ETL pipeline, stored in Supabase PostgreSQL, and visualized through a Dash dashboard.

College football produces large amounts of performance data each season, but that data is often spread across websites and applications. This project centralizes the workflow so users can compare team performance and conference trends from one platform.

Business Problem

Sports statistics are widely available, but users often struggle to compare conference teams, identify performance trends, and visualize team metrics in an interactive way. This project addresses the need for:

- Automated collection of college football records from a REST API
- Structured sports data storage in PostgreSQL
- Interactive visualizations for comparing SEC and Big Ten teams
- Clear KPI reporting and dashboard filtering for non-technical users

Objectives

- Extract 2024 team records from the CollegeFootballData API
- Transform JSON responses into analytics-ready data
- Filter records to SEC and Big Ten teams
- Calculate win percentage from wins and losses
- Validate data quality and load to Supabase PostgreSQL
- Build an interactive Dash dashboard with KPIs, filters, and charts

Technical Approach

The project follows a complete ETL-to-dashboard workflow. Python handles API extraction and transformation, Supabase PostgreSQL stores validated data, and Dash/Plotly provide the dashboard user interface.

Layer	Tool	Role	Output
Extract	requests	Call CollegeFootballData API	Raw JSON
Transform	pandas	Clean, filter, calculate win %	DataFrame
Load	SQLAlchemy	Write data to Supabase PostgreSQL	teams table
Visualize	Dash + Plotly	Create interactive analytics dashboard	Web app

ETL Pipeline and Data Quality Engineering

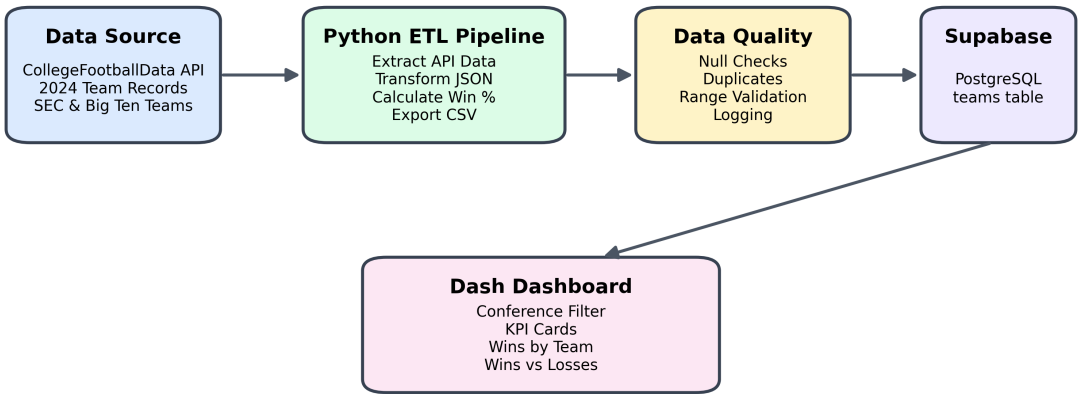
The ETL pipeline extracts 2024 records, transforms them into a clean table, validates the results, loads to PostgreSQL, and exports an analytics-ready CSV.

- Extract: authenticated API call to the records endpoint
- Transform: standardize columns, filter conferences, convert data types, calculate win_percentage
- Load: append validated records and prevent duplicate inserts through incremental loading
- Validate: check nulls, duplicates, invalid types, negative values, and win percentages outside 0-1
- Log: record successful steps and errors in an ETL log file

Technical Architecture Diagram

SEC & Big Ten College Football Analytics Dashboard

End-to-End Data Engineering Architecture



Business Value: Interactive comparison of SEC and Big Ten team performance using automated ETL and live database analytics.

Figure 1. End-to-end architecture from API extraction to Dash dashboard delivery.

Data Source Plan

The primary source is the CollegeFootballData API records endpoint for the 2024 season. The selected data includes team name, conference, wins, and losses. This endpoint supports clear conference and team comparisons while keeping project scope manageable.

Database Schema Documentation

The database uses a simplified analytics-ready schema. A single teams table supports fast dashboard queries while leaving room for future expansion into game-level results, rankings, or multi-season analysis.

Column	Data Type	Constraint	Description
team_name	TEXT	Primary Key	Unique team identifier
conference	TEXT	Required	SEC or Big Ten
wins	INTEGER	Required	Total season wins
losses	INTEGER	Required	Total season losses
win_percentage	NUMERIC	Derived	wins / (wins + losses)

Entity Relationship Diagram

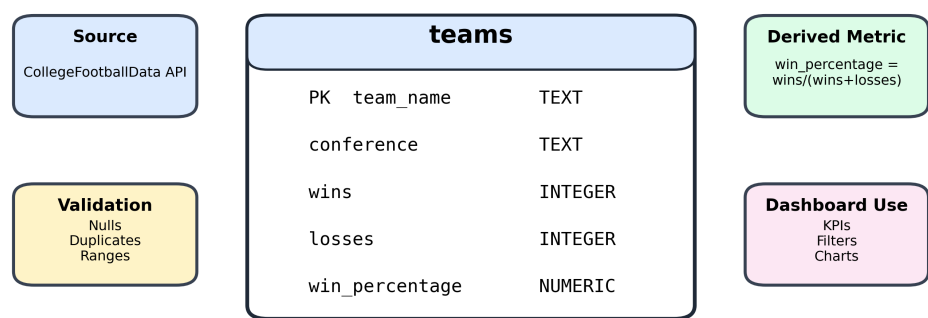


Figure 2. MVP database schema used for the analytics dashboard.

Dashboard Features

The Dash application connects directly to Supabase PostgreSQL and updates visualizations based on the selected conference filter. It is designed for a non-technical audience while demonstrating live database connectivity.

- KPI Cards: Total Teams and Average Win Percentage
- Interactive Filter: All Conferences, SEC, and Big Ten
- Visualization 1: Wins by Team bar chart
- Visualization 2: Wins vs. Losses scatter plot
- User Experience: readable layout, labeled axes, hover details, and intuitive filtering

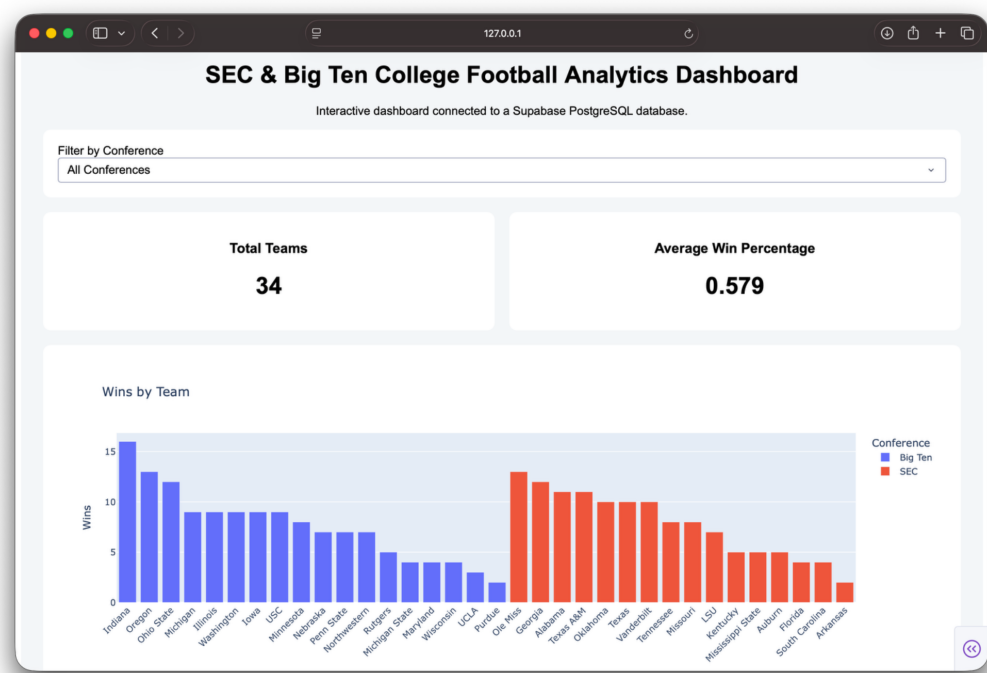


Figure 3. Dash dashboard with KPI cards and team-performance visualization.

Implementation Plan

Week	Tasks
Week 1	Project planning, API research, proposal development
Week 2	ETL extraction and transformation development
Week 3	Supabase integration, schema design, validation framework
Week 4	Dash dashboard, KPI cards, filters, visualizations
Week 5	Testing, documentation, GitHub repository, presentation

Success Metrics

- Successful API extraction and JSON transformation
- Validated database records with no duplicate team rows
- Reliable Supabase loading and dashboard connectivity
- Interactive dashboard performance with clear KPI and chart updates
- Organized GitHub repository with source code, documentation, diagrams, and requirements

Risk Assessment and Mitigation

Risk	Mitigation
API errors or unavailable response	Status-code validation, exception handling, and logging
Missing or malformed records	Null checks, type conversion, and row validation
Duplicate database loads	Incremental loading based on existing team_name values
Dashboard performance issues	Limited MVP scope to SEC and Big Ten teams

Conclusion

This college football dashboard demonstrates a complete end-to-end data engineering workflow using real-world sports data. The project combines REST API extraction, data transformation, quality validation, PostgreSQL storage, and interactive visualization into a practical MVP system. Focusing on SEC and Big Ten teams keeps the project realistic while still producing meaningful insights and a polished analytics deliverable.